NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
FINAL ASSESSMENT FOR
Semester 2 AY2025/2026

CS2030S Programming Methodology II

April 2026 Time Allowed 120 minutes

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains 24 questions and comprises 16 printed pages, including this page.
2. Write all your answers in the answer sheet provided.
3. The total marks for this assessment is 90. Answer **ALL** questions.
4. This is a **CLOSED BOOK** assessment. You are only allowed to refer to one A4-sized cheat sheet during the assessment.
5. All programs in this assessment paper use Java 21 and are compiled with the flags `-Xlint:unchecked` and `-Xlint:rawtypes`.

Multiple Choice Questions (66 points)

For each question below, **shade your answer in the corresponding bubble on the answer sheet.** Each question is worth 3 points. If none of the answers is appropriate, shade X in the answer sheet. If there are multiple correct answers, shade only one of them. Do not shade more than one answer.

1. (3 points) In software engineering, there is a "Law of Demeter", which says that an object should avoid invoking methods of a member object returned by another method. For example, the following code snippet in the `Bank` class violates the Law of Demeter because it invokes the method `startService()` of the `Customer` object returned by the method `nextCustomer()`.

   ```
   Customer c = this.counter.nextCustomer();
   c.startService();
   ```

   Among the statements below, which is the most appropriate reason for following the Law of Demeter?

   A. It ensures that the program follows the Liskov Substitution Principle.

   B. It allows the compiler to perform more rigorous type checks at compile time.

   C. It preserves the abstraction barriers between classes.

   D. It improves performance by reducing the number of method calls.

   E. It improves readability by reducing the number of classes.

   Shade X on the answer sheet if none of the choices above is appropriate.

2. (3 points) In software engineering, there is a principle called the "Open–Closed Principle", which states that software entities should be *open for extension but closed for modification*.

   Consider the following code in the class `PaymentProcessor`:

   ```
   public void process(Payment payment) {
     if (payment instanceof CashPayment) {
       // process cash payment
     } else if (payment instanceof CardPayment) {
       // process card payment
     } else if (payment instanceof PayNowPayment) {
       // process PayNow payment
     }
   }
   ```

   The code violates the open-closed principle, since every time a new type of payment is added, the `process` method must be modified. In order to extend the functionality, we have to modify the existing code.

   Which CS2030S concepts below is most helpful for adhering to the Open–Closed Principle?

   A. Immutability

   B. Tell, Don't Ask

   C. Pure Functions

   D. Polymorphism

   E. Closure

   Shade X on the answer sheet if none of the choices above is correct.

3. (3 points) Which of the following rules is a consequence of adhering to the Liskov Substitution Principle?

   A. Subclasses should not override methods of the superclass.

   B. Subclasses should not implement setters or getters.

   C. Subclasses should not implement additional interfaces that the superclass does not implement.

   D. An overriding method in a subclass should not throw checked exceptions that the overridden method in the superclass does not throw.

   E. An overriding method in a subclass must return the same type as the overridden method in the superclass.

   Shade X on the answer sheet if none of the choices above is correct.

4. (3 points) Suppose we have an interface `I` and an abstract class `A` that implements `I`. Which of the following statements is true?

   A. `I` can only have a single abstract method.

   B. `A` must implement all methods declared in `I`.

   C. `A` cannot be anonymous.

   D. A subclass of `A` cannot implement other interfaces.

   E. A subclass of `A` must override and implement all methods declared in `I`.

   Shade X on the answer sheet if none of the choices above is correct.

Question 5 to Question 8 are based on the following code excerpt:

```
class Maker<T extends Number> {
  static <T extends Number> Maker<T> create() {
    return new Maker<T>();
  }

  T just(T t) {
    // Line X
    return t;
  }
}

class IntegerMaker extends Maker<Integer> {
  Integer just(Integer i) {
    return i;
  }
}
```

5. (3 points) Which of the following method declarations, if added to `IntegerMaker`, will cause a compilation error?

   (i) `Object just(Object n) { return n; }`

   (ii) `Number just(Number n) { return n; }`

   (iii) `Integer just(Number n) { return n; }`

   A. Only (i)

   B. Only (ii)

   C. Only (iii)

   D. Only (i) and (iii)

   E. Only (ii) and (iii)

   Shade X on the answer sheet if none of the choices above is correct.

6. (3 points) Consider the following code snippet:

```
Maker<Integer> m = new IntegerMaker();
Number n = (Number) 0;  // Line Y
m.just(n);              // Line Z
```

   Which of the following statements about the code is correct?

   A. The method `Maker::just(T)` will be invoked.

   B. The method `IntegerMaker::just(Integer)` will be invoked.

   C. Line Y causes a compilation error because `0` cannot be cast to a `Number`.

   D. Line Z causes a compilation error because `Number` is not a subtype of `Integer`.

   E. Line Z causes a compilation error because both `Maker::just(T)` and `IntegerMaker::just(Integer)` are applicable and neither is more specific than the other.

   Shade X on the answer sheet if none of the choices above is correct.

7. (3 points) Which statement below, when inserted into Line X, will NOT cause an unchecked cast compilation warning?

   A. `t = (T) new Object();`

   B. `t = (T) new Integer(0);`

   C. `Maker<T> m = (Maker<T>) new Maker<T>();`

   D. `Maker<T> m = (Maker<T>) new Maker<Number>();`

   E. `Maker<Number> m = (Maker<Number>) new Maker<T>();`

   Shade X on the answer sheet if none of the choices above is correct.

8. (3 points) Which of the following statements compile without errors?

   (i) `Integer i = Maker.create().just(0);`

   (ii) `Integer i = Maker.create().<Integer>just(0);`

   (iii) `Integer i = Maker.<Integer>create().just(0);`

   A. (i) only.

   B. (ii) only.

   C. (iii) only.

   D. (i) and (iii) only.

   E. (i), (ii) and (iii).

   Shade X on the answer sheet if none of the choices above is correct.

9. (3 points) Consider the following method:

```
class Copier {
  static <T> void copyInto(List<? super T> dest, List<? extends T> src) {
    for (T item : src) {
      dest.add(item);
    }
  }
}
```

   Which of the following calls will compile?

   (i) `copyInto(new ArrayList<Number>(), new ArrayList<Integer>())`

   (ii) `copyInto(new ArrayList<Integer>(), new ArrayList<Number>())`

   (iii) `copyInto(new ArrayList<Object>(), new ArrayList<Double>())`

   A. (i) only

   B. (iii) only

   C. (i) and (ii) only

   D. (i) and (iii) only

   E. (i), (ii), and (iii)

   Shade X on the answer sheet if none of the choices above is correct.

10. (3 points) Consider the following code excerpt:

```
interface F {
  int call(Integer t);
}

static F compose(F f, F g) {
  return x -> f.call(g.call(x));
}

F doubler = x -> x * 2;
F adder = x -> x + 3;

compose(doubler, adder).call(1); // Line A
compose(adder, doubler).call(1); // Line B
```

What is the return value at Line A and Line B respectively?

A. 5 and 6.

B. 6 and 5.

C. 5 and 8.

D. 8 and 5.

E. 1 and 1.

Shade X on the answer sheet if none of the choices above is correct.

11. (3 points) Consider the following code excerpt that uses the functional interface `F` defined in the previous question.

```
List<String> log = new ArrayList<>();

F f1 = x -> {
   log.add(x.toString());
   return x;
};

F f2 = x -> x.toString().length();

F f3 = x -> (int)(Math.random() * x);

F f4 = x -> {
  System.out.println(x);
  return x * 2;
};
```

Which of the lambda expressions above are pure functions?

A. `f1`

B. `f2`

C. `f1` and `f2`

D. `f2` and `f3`

E. `f2` and `f4`

Shade X on the answer sheet if none of the choices above is correct.

12. (3 points) Consider the following class:

```
final class ImmutablePoint {
  private final int[] coords;

  ImmutablePoint(int x, int y) {
    this.coords = new int[]{x, y};
  }

  int[] getCoords() {
    return this.coords;
  }
}
```

Which of the following statements correctly describes `ImmutablePoint`?

A. It is immutable because the class is declared `final`.

B. It is immutable because the class is declared `final` and the `coords` field is declared `final`.

C. It is immutable because the `coords` array cannot be resized after construction.

D. It is not immutable because `getCoords()` returns a reference to the internal array.

E. It is not immutable because `int` arrays are always stored on the heap.

Shade X on the answer sheet if none of the choices above is correct.

13. (3 points) Consider the following code excerpt:

```
class R {
  public int foo() {
    return 0;
  }

  public int foo(R r) {
    return 1;
  }
}

interface G {
  int transform(R r);
}

R r = new R();
G g1 = r::foo;
G g2 = R::foo;

g1.transform(r);  // Line C
g2.transform(r);  // Line D
```

What is the return value at Line C and Line D respectively?

A. 0 and 0

B. 0 and 1

C. 1 and 0

D. 1 and 1

E. The code does not compile.

Shade X on the answer sheet if none of the choices above is correct.

14. (3 points) Consider the following code excerpt:

```
class Outer {
  int x = 2;

  static class Inner {
    int x = 3;

    void foo() {
      Producer<Integer> p = () -> x; // Line E
    }
  }

  class C {
    void foo() {
      Producer<Integer> p = () -> x; // Line F
    }
  }
}
```

What is the fully qualified form of the lambda expressions at Line E and Line F respectively?

A. `() -> this.x` and `() -> this.x`

B. `() -> Outer.this.x` and `() -> this.x`

C. `() -> Outer.Inner.x` and `() -> this.x`

D. `() -> Outer.Inner.x` and `() -> Outer.this.x`

E. `() -> this.x` and `() -> Outer.this.x`

Shade X on the answer sheet if none of the choices above is correct.

15. (3 points) Which of the following stream pipelines will always return the list containing the elements [1, 2, 3]?

(i) `Stream.iterate(1, n -> n + 1).limit(3).toList()`

(ii) `Stream.iterate(1, n -> n + 1).takeWhile(n -> n <= 3).toList()`

(iii) `Stream.iterate(1, n -> n + 1).filter(n -> n <= 3).toList()`

A. (i) only

B. (iii) only

C. (i) and (ii) only

D. (ii) and (iii) only

E. (i), (ii), and (iii)

Shade X on the answer sheet if none of the choices above is correct.

16. (3 points) Consider the following code excerpt:

```
List<List<Integer>> nested = List.of(List.of(1, 2, 3), List.of(1, 2, 3), List.of(2, 3, 4));

long count = nested.stream()
  .flatMap(List::stream)
  .distinct()
  .count();
```

What is the value of `count` initialized to?

A. `2`

B. `3`

C. `4`

D. `7`

E. It is not initialized as the code throws an `IllegalStateException`.

Shade X on the answer sheet if none of the choices above is correct.

17. (3 points) Consider the following code excerpt:

```
int foo(int k, int l) {
  return Stream.iterate(k, i -> i <= l, i -> i + 1)
    .filter(i -> isPrime(i))
    .reduce(-1, (a, b) -> {
      if (b - a == 2) {
        System.out.println(a + " " +b);
      }
      return b;
    });
}
```

Assuming `isPrime(i)` correctly returns whether `i` is prime, which of the following correctly describe the method `foo`?

A. It prints the first pair of primes between `k` and `l` (inclusive) with a difference of two and returns the largest prime less than or equal to `l`.

B. It prints the first pair of primes between `k` and `l` (inclusive) with a difference of two and returns the larger of the two primes in the pair.

C. It prints the first pair of primes between `k` and `l` (inclusive) with a difference of two and returns the smaller of the two primes in the pair.

D. It prints all consecutive pairs of primes between `k` and `l` (inclusive) with a difference of two and returns the largest prime less than or equal to `l`.

E. It prints all consecutive pairs of primes between `k` and `l` (inclusive) with a difference of two and returns the smallest prime larger than or equal to `k`.

Shade X on the answer sheet if none of the choices above is correct.

18. (3 points) Suppose `s` is a `Stream<Integer>` that is bounded and non-empty.

Which of the following stream pipeline can be parallelized by adding `.parallel()` call before `.reduce()` without changing the result of the computation?

A. `s.reduce(0, (a, b) -> a - b)`

B. `s.reduce(0, (a, b) -> (a + b) / 2.0);`

C. `s.reduce(0, (a, b) -> a > b ? b : a);`

D. `s.reduce(1, (a, b) -> a + 1);`

E. `s.reduce(1, (a, b) -> b);`

Shade X on the answer sheet if none of the choices above is correct.

19. (3 points) Consider the following code excerpt:

```
CompletableFuture<Integer> result = CompletableFuture
    .<Integer>supplyAsync(() -> { throw new RuntimeException("oops"); })
    .thenApply(x -> x * 2)
    .handle((t, e) -> (e != null) ? -1 : t);

System.out.println(result.join());
```

What does the program print?

A. `0`

B. `null`

C. `oops`

D. `-1`

E. The code throws a `CompletionException`.

Shade X on the answer sheet if none of the choices above is correct.

20. (3 points) Consider the following code excerpt:

```
CompletableFuture<Integer> h = CompletableFuture.supplyAsync(() -> 0);

CompletableFuture<Integer> left = h
  .thenApplyAsync(x -> x + 1)
  .applyToEither(
      h.thenApplyAsync(x -> x - 1),
      x -> x);

CompletableFuture<Integer> right = left.thenApply(x -> x + 1);

CompletableFuture<Integer> cf = left.thenCombine(right, (x, y) -> x + y);

System.out.println(cf.join());
```

What could the program print?

A. either `-1` or `3`
B. either `1` or `2`
C. either `0` or `1`
D. always `1`
E. always `3`

Shade X on the answer sheet if none of the choices above is correct.

Question 21 to Question 22 are based on the following implementation of `Lazy<T>`.

```
interface Producer<T> {
  T produce();
}

interface Transformer<T, R> {
  R transform(T r);
}

public class Lazy<T> {
  private Producer<T> producer;
  private T value;
  private boolean evaluated;

  private Lazy(Producer<T> s) {
    this.producer = s;
    this.evaluated = false;
    this.value = null;
  }

  public static <T> Lazy<T> of(Producer<T> producer) {
    return new Lazy<>(producer);
  }

  public <R> Lazy<R> map(Transformer<? super T, ? extends R> mapper) {
    return new Lazy<R>(() -> mapper.transform(this.get()));
  }

  public T get() {
    if (!this.evaluated) {
      this.value = this.producer.produce();
      this.evaluated = true;
    }
    return this.value;
  }
}
```

21. (3 points) Consider the following code excerpt:

```
Lazy<Integer> x = Lazy.of(() -> {
  System.out.println("produced");
  return 10;
});
x.get();
Lazy<Integer> y = x.map(n -> n + 5);
y.get();
x.get();
```

Which of the following statements is correct?

A. `produced` is printed when `x` is initialized.

B. `produced` is printed when `y` is initialized.

C. `produced` is printed every time `x.get()` is called.

D. `produced` is printed every time `y.get()` is called.

E. `produced` is printed the first time `x.get()` is called.

Shade X on the answer sheet if none of the choices above is correct.

22. (3 points) Consider the following code excerpt:

```
class Counter {
  int x = 0;
  int get() {
    this.x = this.x + 1;
    return this.x;
  }
}

Counter c = new Counter();
Lazy<Integer> lazy = Lazy.of(() -> c.get());

CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> lazy.get());
CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> lazy.get());

CompletableFuture.allOf(f1, f2).join();

c.get(); // Line G
```

Which of the following statements about Line G is correct?

A. It always returns 1, because `lazy` is not evaluated.

B. It always returns 2, because even though both `f1` and `f2` are evaluated, `c.get()` is called once.

C. It always returns 3, because both `f1` and `f2` are evaluated, and each called `c.get()` once.

D. It returns either 1 or 2, depending on the order of evaluation of `f1` and `f2`.

E. It returns either 1, 2, or 3, depending on the order of evaluation of `f1` and `f2`.

Shade X on the answer sheet if none of the choices above is correct.

Short Answer Questions (24 points)

23. (12 points) Refer to the following `InfiniteList` implementation, which is a simplified `Maybe`-less version of `InfiniteList` that does not implement a `filter` method. Please refer to Question 21 for the implementation of `Lazy`.

```
public class InfiniteList<T> {
  private final Lazy<T> head;
  private final Lazy<InfiniteList<T>> tail;

  public static <T> InfiniteList<T> iterate(T seed, Transformer<T, T> next) {
    return new InfiniteList<T>(Lazy.of(() -> seed),
        Lazy.of(() -> InfiniteList.iterate(next.transform(seed), next)));
  }

  private InfiniteList(Lazy<T> head, Lazy<InfiniteList<T>> tail) {
    this.head = head;
    this.tail = tail;
  }

  public T head() {
    return this.head.get();
  }

  public InfiniteList<T> tail() {
    return this.tail.get();
  }

  public <R> InfiniteList<R> map(Transformer<T, R> t) {
    return new InfiniteList<R>(head.map(t), tail.map(x -> x.map(t)));
  }
}
```

In this question, we are interested in implementing two methods of the `InfiniteList` class: `paired` and `accumulate`. Both methods return a new `InfiniteList` that is lazily evaluated, meaning that the elements of the resulting list are not computed until they are needed. Both `paired` and `accumulate` take a curried function as an input. The curried function is of the form `x -> y -> f(x, y)`, where `f` is a binary function that takes two arguments and returns a value.

(a) (8 points) Implement the `paired` method, which takes a curried function `x -> y -> f(x, y)` as the input. `paired` applies the input function to two consecutive elements of the list successively and returns a new `InfiniteList`.

In other words, given a list `[a1, a2, a3, a4, ...]` and a function `f`, the resulting list should be `[f(a1, a2), f(a3, a4), ...]`. This method can be seen as a version of `map`, except that it applies the function to pairs of elements instead of single elements.

For example, suppose the list is `[1, 2, 3, 4, ...]` and the function is `x -> y -> x + y`. This method pairs the elements into (1, 2) (3, 4), . . . and applies the input function, resulting in the list `[3, 7, ...]`.

Fill in the blanks in the following code snippet to implement the `paired` method. Write your answer to `arg1` and `arg2` on the answer sheet.

```
InfiniteList<T> paired(Transformer<T, Transformer<T, T>> f) {
  return new InfiniteList<>(
    __________arg1__________,
    __________arg2__________
  );
}
```

(b) (4 points) Implement the `accumulate` method, which takes a curried function `x -> y -> f(x, y)` as the input. `accumulate` applies the input function to the elements of the list successively and returns a new `InfiniteList`. In other words, given a list `[a1, a2, a3, a4, ...]` and a function `f`, the resulting list should be `[f(a1, a2), f(f(a1, a2), a3), f(f(f(a1, a2), a3), a4), ...]`.

For example, suppose the list is `[1, 2, 3, 4, ...]` and the function is `x -> y -> x + y`, then the resulting list should be `[3, 6, 10, 15,...]`. This method can be seen as a version of `reduce`, except that it returns an `InfiniteList` with intermediate values instead of a single value.

Fill in the blanks in the following code snippet to implement the `accumulate` method. Please note that the `arg1` is the same as the answer to `paired` above. You only need to write `arg3` on the answer sheet.

```
InfiniteList<T> accumulate(Transformer<T, Transformer<T, T>> f) {
  Lazy<T> accHead = __________arg1__________;
  return new InfiniteList<>(
    accHead,
    __________arg3__________
  );
}
```

24. (12 points) Consider the following class:

```
import cs2030s.fp.Transformer;

class Monad<T extends Comparable<T>> {
  private T x;
  private T extreme;

  public static <T extends Comparable<T>> Monad<T> of(T x) {
    return new Monad<T>(x, x);
  }

  private Monad(T x, T extreme) {
    this.x = x;
    this.extreme = extreme;
  }

  public Monad<T> flatMap(Transformer<T, Monad<T>> f) {
    return f.transform(x);
  }
}
```

(a) (6 points) For each of the laws of monad, indicate if `Monad` class satisfies the law or not. If it does not, provide a counterexample where `T` is an `Integer`. If it does, give a brief explanation of why.

Two instances are considered the same if they have the same values of `x` and `extreme`.

(i) Left identity: `Monad.of(x).flatMap(i -> f(i))` is the same as `f(x)`

(ii) Right identity: `m.flatMap(i -> Monad.of(i))` is the same as `m`

(iii) Associativity: `m.flatMap(i -> f(i).flatMap(j -> g(j)))` is the same as `m.flatMap(i -> f(i)).flatMap(j -> g(j))`

(b) (6 points) Now consider an alternative implementation of `flatMap`. The method `max` compares two `T` and returns the "larger" one, based on some order defined by `compareTo`.

```
public static <T extends Comparable<T>> T max(T a, T b) {
  return a.compareTo(b) > 0 ? a : b;
}

public Monad<T> flatMap(Transformer<T, Monad<T>> f) {
  return new Monad<>(f.transform(x).x, max(extreme, f.transform(x).extreme));
}
```

For each of the laws of monad, indicate if the class above satisfies the law or not. If it does not, provide a counterexample where `T` is an `Integer`. If it does, give a brief explanation of why.

(i) Left identity: `Monad.of(x).flatMap(i -> f(i))` is the same as `f(x)`

(ii) Right identity: `m.flatMap(i -> Monad.of(i))` is the same as `m`

(iii) Associativity: `m.flatMap(i -> f(i).flatMap(j -> g(j)))` is the same as `m.flatMap(i -> f(i)).flatMap(j -> g(j))`

END OF PAPER

Appendix: Selected Java API Reference

This appendix is provided for reference during the examination. The type parameter `T` refers to the element type of the stream or future.

`Stream<T>` (java.util.stream)

Creation (static methods on `Stream`)

`Stream.of(T... values)`	Returns a sequential stream of the given elements.
`Stream.iterate(T seed, UnaryOperator<T> f)`	Infinite stream seeded with `seed`; each element produced by `f`.
`Stream.generate(Supplier<T> s)`	Infinite stream where each element is produced by `s`.

Intermediate operations (return a new `Stream`)

`filter(Predicate<T> pred)`	Keeps elements for which `pred` is true.
`map(Function<T,R> mapper)`	Transforms each element; returns `Stream<R>`.
`flatMap(Function<T,Stream<R>> mapper)`	Transforms and flattens; returns `Stream<R>`.
`limit(long n)`	Truncates to at most `n` elements.
`skip(long n)`	Discards the first `n` elements.
`takeWhile(Predicate<T> pred)`	Takes elements while `pred` holds; stops at first false.
`distinct()`	Removes duplicate elements (using `equals`).
`sorted()`	Returns elements in natural ascending order.

Terminal operations (consume the stream; stream cannot be reused afterwards)

`forEach(Consumer<T> action)`	Performs `action` on each element.
`toList()`	Collects elements into an unmodifiable `List<T>`.
`count()`	Returns the number of elements as a `long`.
`reduce(T identity, BinaryOperator<T> acc)`	Reduces to a single `T` using `acc`, starting from `identity`.
`reduce(U identity, BiFunction<U, T, U> acc, BinaryOperator<T> comb)`	Reduces to a single `T` using `acc` and `comb`, starting from `identity`.
`anyMatch(Predicate<T> pred)`	`true` if any element matches `pred`.
`allMatch(Predicate<T> pred)`	`true` if all elements match `pred`.
`noneMatch(Predicate<T> pred)`	`true` if no element matches `pred`.

`CompletableFuture<T>` (java.util.concurrent)

Creation (static factory methods)

`CompletableFuture.completedFuture(T value)`	Returns an already-completed future holding `value`.
`CompletableFuture.supplyAsync(Supplier<T> s)`	Runs `s` asynchronously; returns `CompletableFuture<T>`.
`CompletableFuture.allOf(CompletableFuture<?>... cfs)`	Returns a `CompletableFuture<Void>` that completes when all given futures complete.

Chaining (instance methods)

`thenApply(Function<T,U> f)`	Like `map`: applies `f` to the result; returns `CompletableFuture<U>`.
`thenApplyAsync(Function<T,U> f)`	Like `thenApply` but executes `f` asynchronously.
`thenCompose(Function<T,CompletableFuture<U>> f)`	Like `flatMap`: applies `f` and flattens; returns `CompletableFuture<U>`.
`thenAccept(Consumer<T> action)`	Runs `action` with the result; returns `CompletableFuture<Void>`.
`thenCombine(CompletableFuture<U> other, BiFunction<T,U,V> f)`	When both futures complete, applies `f` to both results; returns `CompletableFuture<V>`.
`applyToEither(CompletableFuture<T> other, Function<T,U> f)`	Applies `f` to whichever of the two futures completes first; returns `CompletableFuture<U>`.

Error handling

`handle(BiFunction<T,Throwable,U> f)`	Called whether or not an exception occurred; exactly one of `T` and `Throwable` is non-null. Returns `CompletableFuture<U>`.
`exceptionally(Function<Throwable,T> f)`	Called only on exception; returns `CompletableFuture<T>` with the recovered value.

Retrieving the result

`T join()`	Blocks until complete; throws `CompletionException` (unchecked) on failure.

This page is intentionally left blank.